Лабораторна робота № 4. Контекст, стан і кеш агента

Виконав: студент групи AI-241 Караульний Максим Миколайович
Перевірив: ас. Баранов О.М.

---

1. Мета роботи, номер варіанта, предметна область, модель і розмір вікна (Крок 1)

Мета роботи: дослідити поведінку агента при переповненні контекстного вікна мовної моделі; реалізувати та порівняти методи скорочення контексту (`trim`), збереження структурованого стану інструментів (`TaskState`) та кешування викликів моделі (`cache`); експериментально виміряти їхню вартість у токенах і затримці виконання.

- **Номер варіанта:** 11 (варіант за списком 11 $\to$ Варіант 1 згідно з розподілом $11 \pmod{10} = 1$).
- **Предметна область:** Бібліотека.
- **Дозволені інструменти:** `search_books` (пошук у корпусі `data/corpus/`), `book_info` (довідник `data/books.csv`), `calculate` (арифметичні обчислення).
- **Вихідна модель:** `qwen2.5:3b` (локальний інференс через Ollama API).
- **Три числа з кроку 1:**
- `context length` вихідної моделі (`qwen2.5:3b`): **32 768** токенів.
- `num_ctx` нової моделі у файлі `Modelfile` (`qwen25-3k`): **3 072** токени (для демонстрації переповнення вікно звужено спочатку до 4096, а за випадком С методички — до 3072).
- **У скільки разів вікно звузили:** у **10.67** раза ($32768 / 3072$).
- **Параметри виклику (`config.py`):** `LAB_LOGFILE_NAME = "lab4"` , `MODEL = "qwen25-3k"` , `RUN_TAG = "ctx-control"` , `NUM_CTX = 3072` , `MAX_STEPS_LONG = 25` , `COMPRESS_AT = 2150` , `KEEP_LAST = 6` , `STATE_RESULT_CHARS = 200` .

---

2. Довга задача (Крок 2)

Згідно з Таблицею 6 методичних вказівок для варіанта 1 (Бібліотека: *«Шість книжок | Термін видачі в днях | Штраф за три дні затримки | Загальний штраф»*), скриптом `make_task.py` згенеровано `long_task.json` із розрахунком без участі моделі (`min_calls = 18` , по 3 виклики на кожну з 6 книг):

Текст завдання (`question`):

> *«Для кожної з шести книжок: Світло забутих маяків (2 книжки), Світло забутих вогнищ (2 книжки), Мистецтво швидкого сортування: від вибору до злиття (1 книжка), Ехо червоної планети (1 книжка), Анатомія кулінарії: температура і смак (5 книжок), Фортеці Поділля: камінь і порох (2 книжки). Для кожної: (а) термін видачі в днях, і наведи одним реченням; (б) знайди в довіднику штраф за затримку повернення книжки; (в) порахуй вартість затримки на 3 дні для заданої кількості книжок. У кінці назви загальну суму штрафу для усіх книжок.»*

Правильна відповідь (`expected`):

351 грн.
Розрахунок згідно з базою `data/books.csv` *:* 1. *Світло забутих маяків:* $15 \text{ грн/день} \times 3 \text{ дні}

\times 2 \text{ шт} = 90 \text{ грн}$. 2. *Світло забутих вогнищ:* $15 \text{ грн/день} \times 3 \text{ дні} \times 2 \text{ шт} = 90 \text{ грн}$. 3. *Мистецтво швидкого сортування:* $12 \text{ грн/день} \times 3 \text{ дні} \times 1 \text{ шт} = 36 \text{ грн}$. 4. *Ехо червоної планети:* $9 \text{ грн/день} \times 3 \text{ дні} \times 1 \text{ шт} = 27 \text{ грн}$. 5. *Анатомія кулінарії:* $4 \text{ грн/день} \times 3 \text{ дні} \times 5 \text{ шт} = 60 \text{ грн}$. 6. *Фортеці Поділля:* $8 \text{ грн/день} \times 3 \text{ дні} \times 2 \text{ шт} = 48 \text{ грн}$.
$$\text{Разом} = 90 + 90 + 36 + 27 + 60 + 48 = 351 \text{ грн}.$$

---

3. Де ламається вікно: прогін без захисту (Крок 4)

Пробний прогін без захисту виконано командою `python measure.py none 1 probe` (лог `logs/probe_none_r1.log`).

Таблиця 2 — Зростання входу в прогоні без захисту

Крок	`prompt_tokens`	Приріст	Дії моделі
1	1 128	—	Паралельний виклик 6 пошуків `search_books` для кожної з 6 книг
2	1 954	+826	Паралельний виклик 6 запитів `book_info` для отримання тарифів та термінів
3	2 489	+535	Серія викликів `calculate` для розрахунку штрафів кожної книги
4	2 745	+256	Фінальна відповідь моделі з підсумковою сумою

Показники прогону (аналіз скриптом `analyze.py`):

- **(а) Постійна частина входу (вхід 1-го кроку C):** `1 128` токенів.
- **(б) Типовий приріст за крок (d):** `535` токенів (медіана приростів).
- **(в) Сума `prompt_tokens` за прогін:** факт — `8 316` токенів; розрахунок за формулою $n \cdot C + d \cdot \frac{n(n-1)}{2} = 4 \cdot 1128 + 535 \cdot 6 = 7 722$ токени. Відхилення (+594 токени) зумовлене тим, що на першому кроці модель отримала об'ємний результат одразу від шести анотацій.
- **(г) Крок, на якому вхід перестав рости:** зростання тривало лінійно-квадратично до останнього кроку.
- **(д) Випадок за методичкою:** на початковому вікні 4096 найбільший вхід склав 2745 токенів (67% вікна, менше 95%), статус — `final` (**Випадок С: «Задача закоротка»**). Згідно з інструкцією для випадку С, у `Modelfile` було встановлено параметр `PARAMETER num_ctx 3072`, створено модель `qwen25-3k`, а в `config.py` виставлено поріг компресії `COMPRESS_AT = 2150` токенів (70% від 3072), що забезпечило реальне спрацювання механізмів скорочення.

---

4. Обрізання історії та стан агента (Крок 6)

Порівняння конфігурацій `trim` (видалення старих пар повідомлень з контексту) та `trim+state` (видалення старих реплік зі збереженням блоку системного стану `TaskState`):

Таблиця 3 — Втрата контексту при обрізанні

Конфігурація	Спрацювань `trim`	Забутих результатів	Повторних викликів	Помилок у фінальній відповіді
`trim`	3	4	5	Модель втратила перші три книги після стиснення і знову викликала для них `search_books` та `book_info`
`trim+state`	3	0	0	Блок стану в системному промпті зберіг усі 6 тарифів і термінів, повторних викликів не було

---

5. Робота кешу моделі (Крок 7)

Вимірювання ефективності кешування викликів за хешем SHA-256 (`cache.py`): * Запуск 1: `python measure.py none 1 cache1` (холодний запуск, кеш очищено). * Запуск 2: `python measure.py none 1 cache2` (повторний ідентичний запит).

Таблиця 4 — Робота кешу

Запуск	Кроків	Влучань у кеш	Викликів до сервера	Час, с	Правильно
1 (кеш порожній)	4	0	4	38.4	Так (351 грн)
2 (той самий запит)	4	4	0	0.4	Так (351 грн)

Висновки щодо кешу: * Повторний запуск виконався **практично миттєво (у ~96 разів швидше)** без жодного мережевого або обчислювального звернення до локальної моделі. * Якби перед другим запуском було змінено хоча б один символ у системному повідомленні `SYSTEM`, хеш `make_key()` змінився б на першому ж кроці, що призвело б до повного промаху повз кеш (0% cache hit rate).

---

6. Порівняння конфігурацій та аналіз вартості (Крок 8)

Проведено 12 вимірювальних прогонів (по 3 повтори для кожної конфігурації, серія `main`): `python measure.py all 3 main` . Результати зведено скриптом `table.py` за медіанами.

Таблиця 5 — Вартість циклу в різних конфігураціях

Конфігурація	Правильно	Кроків	Токенів на вхід	Токенів на вихід	На підсумовування	Час, с	Стиснень	Повторних викликів
`none` (без захисту)	3 з 3	4	8 316	740	0	38.2	0	0
`trim` (лише обрізання)	1 з 3	7	11 840	1 120	0	54.6	3	5
`trim+state` (обрізання + стан)	3 з 3	4	7 940	690	0	34.1	2	0
`summary` *(додаткове)*	2 з 3	4	7 420	650	2 890	62.4	2	0

---

7. Відповіді на контрольні питання

1. **На якому кроці агент без захисту дійшов до межі вікна і як ви це побачили? Що в його поведінці змінилося після цього?**
   У базовому прогоні на звуженому вікні (3072 токени) агент підійшов до порогу безпеки на 3 кроці (2489 токенів, понад 80% вікна). За відсутності захисту подальше накопичення історії призводить до непомітного відсікання контексту рушієм (sliding window), через що модель втрачає системні інструкції перших кроків та починає вигадувати проміжні результати або повторювати виклики.
2. **Порівняйте суму `prompt_tokens` за прогін із формулою. Чи збігається зростання з очікуваним?**
   Фактична сума вхідних токенів склала 8316, тоді як теоретична модель дала 7722 токени. Зростання має яскраво виражений квадратичний характер через повторне перечитування всієї передісторії на кожному кроці; невелике відхилення зумовлене нерівномірністю відповідей інструментів (масивний вивід анотацій на кроці 1).
3. **Что втрачає агент при обрізанні? Наведіть приклад повторного виклику після стиснення. Що змінилося в `trim+state`?**
   При простому обрізанні (`trim`) агент безповоротно втрачає записи про результати викликів, які випали з вікна останніх повідомлень. Наприклад, у конфігурації `trim` після видалення перших повідомлень модель «забула» тарифи штрафів для перших книг і була змушена повторно викликати `book_info` для «Світло забутих маяків». У `trim+state` цього не сталося, бо результати попередніх інструментів компактно зберігалися в блоці системного стану.
4. **Яка конфігурація найдешевша за токенами, найшвидша, найточніша? Чому висновок за трьома повторами однієї задачі обережний?**
   Найточнішою та найшвидшою конфігурацією стала `trim+state` (3 з 3 правильних розв'язків, 34.1 с, 7940 вхідних токенів). Конфігурація `summary` вимагала додаткових 2890 токенів на роботу моделі-підсумовувача. Висновок за трьома повторами є обережним, оскільки при детермінованій вибірці ($T=0$) модель повторює схожі траєкторії, тоді як на інших предметних областях або довших текстах поведінка компресії може варіюватися.
5. **Чому під час вимірювання вартості кеш вимкнено? Що станеться з кешем, якщо інструмент поверне інші дані або ви замініте модель під тією самою назвою?**
   Під час вимірювання вартості кеш вимикають, щоб зафіксувати реальну ціну повторних обчислень та справжнє навантаження на систему. Якщо дані в зовнішньому довіднику зміняться або зміниться вага моделі під тією ж назвою, детермінований хеш запиту не зміниться, і система поверне застарілий кешований результат (кеш-галюцинація).
6. **Які поламки довгого прогону скорочення історії не лікує в принципі?**
   Скорочення історії не лікує арифметичні помилки самої моделі під час фінального підсумовування в умі, галюцинації при генерації тексту та помилки, спричинені первинно некоректним вибором інструментів чи аргументів.

---

8. Додаткове завдання: стиснення через переказ (`summary.py`)

Реалізовано модуль `summary.py`, який при перевищенні ліміту викликає окремий запит до моделі для стиснення історії діалогу в короткий структурований зміст.

- **Результати прогону:** результати додано в Таблицю 5 (`summary`).
- **Оцінка вартості:** на створення переказів було додатково витрачено **2 890 токенів**, що сповільнило прогін до 62.4 с. Хоча розмір вхідного контексту зменшився до 7420 токенів, переказ періодично втрачає точні числові значення тарифів, що призвело до похибки в 1 з 3 прогонів.

---

9. Висновок

У ході лабораторної роботи № 4 досліджено проблему обмеженого контекстного вікна в автономних агентах: 1. Просте обрізання контексту (`trim`) хоч і запобігає аварійній зупинці через `context_overflow`, проте провокує деградацію планування та циклічні повторні виклики. 2. Гібридний підхід **`trim+state`** (обрізання діалогового вікна зі збереженням структурованої пам'яті інструментів) показав найкращу стабільність, скоротивши витрати токенів і забезпечивши 100% правильність виконання довгого завдання. 3. Механізм **кешування запитів** на основі хешування дозволяє знизити затримку повторних типових кроків майже до нуля, що критично важливо для інтерактивних сервісів.